TrustMesh — End-to-End Build Plan

1-Month MVP · 3-Developer Team (1 Frontend, 2 Backend + Blockchain)

---

0. Team Split & Ownership

Dev	Role	Owns
Dev A	Frontend	React app, wallet/Web3 integration, all screens, UI state, calling backend + contract
Dev B	Backend Lead	Express API, MongoDB, OTP service, document validation, hashing, auth/roles
Dev C	Blockchain Lead	Solidity contract, Ganache/Hardhat setup, contract tests, Web3 event listeners, deployment scripts, and backend support (verification engine, trust score logic)

Dev B and Dev C both touch the backend repo but split cleanly: **Dev B = off-chain data & verification logic**, **Dev C = on-chain contract & chain-sync logic**. They pair on the `/verification` endpoints since that's the seam between the two.

Stand-up cadence: 15 min daily + Friday integration/demo checkpoint each week.

1. Detailed UI Plan for Frontend Dev (Dev A)

Build with **React + Vite, TailwindCSS, ethers.js (or Web3.js), React Router, Axios, React Query/Zustand** for state.

1.1 Design Direction

- Tone: "credential-grade trust" — clean, neutral, slightly technical (think a hybrid of LinkedIn's clarity + a blockchain explorer's precision).
- Palette: deep navy/charcoal primary (`#0F172A`), one accent (electric blue or teal, `#2563EB` / `#0EA5E9`) for verified states, plus semantic colors: green = VERIFIED, amber = PENDING, red = REJECTED/REVOKED, gray = SELF-CLAIMED.
- Typography: one clean sans (Inter or IBM Plex Sans) — headings semibold, body regular, monospace font (JetBrains Mono / Roboto Mono) specifically for wallet addresses, tx hashes, and evidence hashes so they read as "cryptographic" data.
- Every claim/status anywhere in the UI must carry a colored badge (`SELF-CLAIMED` / `PENDING` / `EMAIL-VERIFIED` / `EVIDENCE-VALIDATED` / `VERIFIED` / `REJECTED` / `REVOKED`) — this badge system is the visual backbone of the whole app.

1.2 Screen-by-Screen Spec

1. Landing / Connect Wallet

- Full-screen hero: product name, one-line pitch, "Connect Wallet" primary button (MetaMask icon).
- On connect: detect network, prompt switch to Ganache/local network if mismatched, show short wallet address (`0x1234…abcd`) top-right once connected.
- If wallet has no profile yet → redirect to "Create Profile"; if it does → redirect to Dashboard.

2. User Profile (Create/Edit)

- Form: name, headline, bio, education (repeatable rows), skills (tag input), experience (repeatable rows), profile image upload.
- Email field + "Verify Email" button opening the OTP modal (6-digit input, countdown timer, resend-after-cooldown).
- Save state indicators (saving/saved) — no full-page reload on save.

3. Create Claim

- Stepper form (3 steps):

  1. **Claim type** (Skill / Project / Experience / Contribution / Achievement / Recommendation) — card-select UI.

  2. **Details** — title, description, dynamic fields depending on type (e.g., organization name + domain for Experience).

  3. **Evidence** — drag-and-drop file upload (PDF/PNG/JPG/JPEG, 5MB max), live client-side validation (type/size) before submit, progress bar, then show computed SHA-256 hash back to the user once server confirms.

- Submit → claim enters `DRAFT` → `SUBMITTED` → `EVIDENCE_CHECK` with a visible progress tracker (mini status stepper matching the lifecycle: DRAFT → SUBMITTED → EVIDENCE_CHECK → VERIFICATION_PENDING → VERIFIED/REJECTED).

4. Verification Requests (My Claims)

- Table/list of the user's own claims, each row: type, title, status badge, date, "View" link.
- Filter/sort by status. Empty state for new users.

5. Verifier Dashboard (only visible to `role: verifier`)

- Queue of `VERIFICATION_PENDING` claims assigned/visible to this verifier.
- Each row expands to claim detail + evidence preview (PDF/image inline viewer) + Approve/Reject buttons.
- Approve → opens MetaMask transaction modal (native MetaMask popup, app just shows "Confirm in MetaMask…" loading state) → on tx confirmation, show success toast with tx hash link.

- Reject → reason text field (stored off-chain) → status updates immediately, no blockchain tx needed for rejection (per data model, only key events go on-chain — confirm exact policy with Dev C, but keep reject light-weight/off-chain to save gas/dev time unless the doc requires on-chain rejection, which the contract functions list does include `rejectClaim()`, so route Reject through the contract too for consistency).

6. Claim Details & Blockchain Proof (the most important screen — public-facing)

- Header: claim title, type, status badge (large).
- "Self-Claimed" section vs "Verified" section — visually separated (this is the doc's key design principle, make it literal in the UI: two clearly divided panels).
- Evidence panel: filename, SHA-256 hash (monospace, copyable), uploaded date.
- Verification panel (only if verified): verifier name/wallet, timestamp, **transaction hash with link to a local block explorer or raw Ganache tx display**, "View on chain" expandable raw event data.
- Shareable public URL for recruiters (no login required to view).

7. Verification History

- Chronological timeline (vertical) of all verification events for a profile: created, evidence-checked, verified, revoked — timestamps + actor (wallet) on each node.

8. Trust / Reputation Dashboard

- Big score number + breakdown by claim/source (Organization=10, Mentor/Client=8, Peer=3, Self=0 — bar or radial chart).
- List of contributing claims with their point value, and a note explaining "this is a summary score; verification records are the actual evidence" (transparency copy required by design principle).

9. Admin / Verifier Management (role: admin)

- Table of registered verifiers, wallet address, status (authorized/pending), "Register Verifier" form (address + name), "Revoke Authorization" action — both go through the contract's `registerVerifier()`/`authorizeVerifier()`.

1.3 Frontend Component/Folder Plan

```
client/src/
├── components/      # StatusBadge, ClaimCard, EvidenceUploader, OtpInput, WalletConnectButton,
│                    TxStatusModal, Timeline, TrustScoreChart, VerifierQueueRow
├── pages/           # Landing, Profile, CreateClaim, MyClaims, VerifierDashboard,
│                    ClaimDetails, VerificationHistory, TrustDashboard, AdminVerifiers
├── contracts/       # TrustMesh ABI + contract address config (per network)
├── services/        # api.js (axios instance), web3.js (ethers provider/signer/contract), auth.js
├── hooks/           # useWallet, useClaim, useContractEvents, useOtp
└── utils/           # formatAddress, formatHash, statusColor, fileValidators
```

1.4 Frontend Build Order (what Dev A should build first)

1. Wallet connect + network detection (unblocks everything else)
2. StatusBadge + design tokens/theme (used everywhere)
3. Profile create/edit + OTP modal (needs Dev B's `/auth` and `/profile` endpoints — mock with static JSON until ready)
4. Create Claim flow + evidence upload
5. Claim Details / Blockchain Proof page (needs Dev C's ABI + deployed address)
6. Verifier Dashboard + MetaMask approve/reject flow
7. History, Trust Dashboard, Admin — polish last

---

2. Full Backend Plan (Dev B, Node.js + Express + MongoDB)

2.1 Stack

Express, Mongoose, `jsonwebtoken` (session/role auth using wallet-signed message, not passwords), `multer` (uploads), `crypto` (SHA-256), `nodemailer` (OTP email), `express-rate-limit`, `helmet`, `cors`, `dotenv`.

2.2 Auth Model (wallet-based, no passwords)

- Login = **sign-a-message flow**: backend issues a nonce for a wallet address → user signs it in MetaMask → backend verifies signature (`ethers.verifyMessage`) → issues JWT containing `{ walletAddress, role }`.
- Role-based middleware: `requireRole('verifier')`, `requireRole('admin')`, applied per route.

2.3 Mongoose Schemas (from the doc's data model)

```
// User
{ walletAddress, name, email, emailVerified, role: ['owner','verifier','admin'], createdAt, updatedAt }

// Profile
{ userId, headline, bio, education: [], skills: [], experience: [], profileImage, createdAt }

// Claim
{ profileId, type, title, description, evidenceUrl, evidenceHash, status: [
'DRAFT','SUBMITTED','EVIDENCE_CHECK','VERIFICATION_PENDING','VERIFIED','REJECTED'
,'REVOKED'
  ], verification: { verifierId, verifierWallet, verificationMethod, verifiedAt, transactionHash, status },
  createdAt, updatedAt }

// Verification
{ claimId, verifierId, verifierWallet, method, status, transactionHash, blockchainTimestamp, createdAt
}

// OtpVerification
{ userId, email, otpHash, expiresAt, attempts, verified }
```

2.4 API Endpoints (extends the doc's table with full detail)

Method	Route	Auth	Purpose
POST	/api/auth/nonce	none	Issue nonce for wallet signature login
POST	/api/auth/login	none	Verify signature, issue JWT
POST	/api/auth/send-otp	owner	Generate OTP, hash+store, email it

POST	/api/auth/verify-otp	owner	Compare hash, set `emailVerified=true`
POST	/api/profile	owner	Create profile
GET	/api/profile/:wallet	public	Get public profile
PUT	/api/profile	owner	Update profile
POST	/api/claims	owner	Create claim (status DRAFT→SUBMITTED)
GET	/api/claims/:id	public	Get single claim + verification info
GET	/api/claims?profileId=	public	List claims for a profile
POST	/api/claims/:id/evidence	owner	Upload evidence file (multer), run validation
POST	/api/verification/hash	owner/system	Compute & store SHA-256 for uploaded evidence
POST	/api/verification/evidence-check	system	Run automated evidence checks, move status to EVIDENCE_CHECK/VERIFICATION_PENDING
POST	/api/verification/request	owner	Mark claim VERIFICATION_PENDING, notify verifier
POST	/api/verification/sync	verifier/backend listener	Record on-chain tx result (called after contract event fires) into `verifications` + update claim status
GET	/api/verifiers	public	List authorized verifiers
POST	/api/verifiers	admin	Register verifier off-chain record (mirrors on-chain `registerVerifier`)
GET	/api/trust-score/:profileId	public	Compute/return trust score

2.5 Verification Engine (core backend logic, owned by Dev B, reviewed by Dev C)

Implements the doc's rule:

```
IF email_verified AND evidence_valid AND verifier_authorized AND verifier_approved
THEN claim_status = VERIFIED ELSE PENDING
```

- services/evidenceValidator.js: file type/size/MIME/signature check, basic metadata/reference matching, name-match against profile.
- services/hashService.js: SHA-256 generation + duplicate-hash lookup.
- services/otpService.js: generate, hash (bcrypt/sha256), 5-min expiry, attempt limiter, resend cooldown, invalidate-previous-on-new.
- services/trustScoreService.js: sum points by verification source (Organization=10, Mentor/Client=8, Peer=3, Self=0), apply caps/duplicate-verifier rules, return breakdown for the dashboard.

2.6 Blockchain Sync Layer (Dev C, lives in backend)

- services/web3Listener.js: Ethers.js provider listening to Ganache for ClaimVerified, ClaimRejected, VerificationRevoked, VerifierRegistered events → writes to verifications collection and updates claims.status + transactionHash so the off-chain data always mirrors the chain (source of truth for status = chain event, not the frontend call).
- This decouples "user clicked approve in MetaMask" from "DB is updated" — DB updates only after the event is actually mined, avoiding inconsistent states if a tx fails/is rejected in MetaMask.

2.7 Backend Folder Structure

```
server/src/
├── controllers/    authController, profileController, claimController, verificationController, verifierController
├── routes/         auth.js, profile.js, claims.js, verification.js, verifiers.js
├── models/         User, Profile, Claim, Verification, Otp
├── services/       otpService, hashService, evidenceValidator, trustScoreService, web3Listener, emailService
├── middleware/     auth (JWT verify), requireRole, rateLimiter, uploadMulter, errorHandler
└── utils/          signatureVerify, constants (statuses, claim types)
```

3. Full Solidity / Blockchain Plan (Dev C)

3.1 Toolchain

Hardhat (recommended over raw Truffle for speed of iteration) + Ganache (local chain) + Ethers.js + Mocha/Chai for tests + dotenv for private key/RPC config.

3.2 TrustMesh.sol — Contract Design

State:

```
enum ClaimStatus { PENDING, VERIFIED, REJECTED, REVOKED }

struct Claim {
    uint256 id;
    address claimant;
    string claimType;      // or bytes32 for gas savings
    string referenceHash;  // off-chain claim/evidence pointer (SHA-256 / MongoDB ref)
    ClaimStatus status;
    address verifier;
    uint256 timestamp;
}

mapping(uint256 => Claim) public claims;
mapping(address => uint256[]) public userClaims;
mapping(address => bool) public authorizedVerifiers;
address public owner;
uint256 public claimCounter;
```

Core functions (mirroring the doc's list):

```
function registerProfile() external;                                  // marks wallet as a registered profile owner
function createClaim(string calldata claimType, string calldata referenceHash) external returns (uint256);
function requestVerification(uint256 claimId) external;               // claimant-only, moves off-chain trigger
function verifyClaim(uint256 claimId) external onlyAuthorizedVerifier;  // emits ClaimVerified
function rejectClaim(uint256 claimId, string calldata reason) external onlyAuthorizedVerifier;
function revokeVerification(uint256 claimId) external onlyAuthorizedVerifier; // does NOT delete history, sets REVOKED + emits event
```

```
    function getClaim(uint256 claimId) external view returns (Claim memory);
    function getUserClaims(address user) external view returns (uint256[] memory);
    function getVerificationHistory(uint256 claimId) external view returns (...); // via events off-chain, or
on-chain array if needed
    function registerVerifier(address verifier) external onlyOwner;
    function authorizeVerifier(address verifier) external onlyOwner;
```

Modifiers:

```
    modifier onlyOwner() { require(msg.sender == owner, "Not owner"); _; }
    modifier onlyAuthorizedVerifier() { require(authorizedVerifiers[msg.sender], "Not authorized
verifier"); _; }
    modifier onlyClaimant(uint256 claimId) { require(claims[claimId].claimant == msg.sender, "Not
claimant"); _; }
```

Events (exact, from the doc):

```
    event ClaimCreated(uint256 indexed claimId, address indexed claimant, uint256 timestamp);
    event ClaimVerified(uint256 indexed claimId, address indexed claimant, address indexed verifier,
uint256 timestamp);
    event ClaimRejected(uint256 indexed claimId, address indexed claimant, address indexed verifier,
string reason, uint256 timestamp);
    event VerificationRevoked(uint256 indexed claimId, address indexed verifier, uint256 timestamp);
    event VerifierRegistered(address indexed verifier, uint256 timestamp);
```

Key on-chain vs off-chain discipline (per doc's Table 3):

- On-chain: claim/reference ID, claimant wallet, verifier wallet, claim type, status, timestamp, evidence hash.
- Off-chain (Mongo/S3-style uploads dir): profile bio, description text, large files, images, OTP/email data, UI metadata.
- Keep `referenceHash` as a short string (the SHA-256 hex or a compact IPFS-style pointer) — never push full documents/descriptions on-chain (gas cost + doc's explicit rule).

3.3 Access Control Design

- Only `owner` (deployer/admin wallet) can register/authorize verifiers.
- Only `authorizedVerifiers[msg.sender] == true` can call `verifyClaim/rejectClaim/revokeVerification`.
- A normal user calling `verifyClaim()` must revert — this needs an explicit negative test (doc section 18 flags this directly).
- Never trust the backend alone to gate this — the contract enforces it independently, backend is just a UX/indexing layer.

3.4 Deployment/Migration Plan

- `hardhat.config.js` targeting local Ganache RPC (`http://127.0.0.1:7545` or Hardhat's own local node if Ganache GUI isn't required).
- `scripts/deploy.js`: deploys `TrustMesh.sol`, logs address + ABI path, auto-writes `client/src/contracts/TrustMesh.json` (address+ABI) so frontend never has to manually copy it.
- `.env.example`: `RPC_URL`, `PRIVATE_KEY` (Ganache test account), `CHAIN_ID`.

3.5 Solidity Test Plan (Mocha/Chai + Hardhat)

- Deploy fixture per test file.
- `createClaim` — succeeds, increments counter, emits `ClaimCreated`.
- `verifyClaim` — succeeds only from authorized verifier, emits `ClaimVerified`, status becomes VERIFIED.
- `verifyClaim` from unauthorized wallet — **must revert**.
- `rejectClaim` — status REJECTED, emits `ClaimRejected`.
- `revokeVerification` — status REVOKED, previous event history still queryable (doesn't erase past events — check via `queryFilter` on past blocks).
- Duplicate verification attempt on an already-VERIFIED claim — should revert or no-op per your chosen rule (decide with Dev B before Week 3).
- Invalid claim ID (never created) — reverts on all read/write functions.
- `registerVerifier/authorizeVerifier` — only callable by `owner`, reverts otherwise.

4. Weekly Execution Plan — 3 Devs, 4 Weeks

Week 1 — Foundation (all 3 devs, mostly parallel setup)

Day	Dev A (Frontend)	Dev B (Backend)	Dev C (Blockchain)
Mon	Repo scaffold (Vite+React+Tailwind), routing skeleton, design tokens/theme	Express app skeleton, Mongoose connection, .env setup, folder structure	Hardhat project init, Ganache running locally, base TrustMesh.sol file with structs/enums
Tue	Wallet connect component (ethers.js provider/signer), network-detect logic	User/Profile/Claim/Verification/OTP Mongoose schemas	Implement registerProfile, createClaim, getClaim, getUserClaims
Wed	StatusBadge + shared UI components, layout shell (nav, role-aware routing)	Auth: nonce endpoint + signature-verify login + JWT middleware	Implement registerVerifier, authorizeVerifier, access-control modifiers
Thu	Landing page + connect flow wired to real wallet	Role middleware (requireRole), basic error handler, rate limiter setup	Implement verifyClaim, rejectClaim, revokeVerification + all 5 events
Fri	**Integration checkpoint**: wallet connect → mock login working end to end	Auth flow demo: sign message → JWT issued and verified	Local deploy script working, ABI+address exported; write first 3 Hardhat tests

Week 1 Deliverable: Working local blockchain architecture — contract deployed on Ganache, backend auth via wallet signature, frontend can connect wallet and log in.

Week 2 — Profiles + Claims

Day	Dev A	Dev B	Dev C
Mon	Profile create/edit form + image upload UI	/api/profile CRUD endpoints	Finish + review remaining contract functions; write full unit test suite (create/verify/reject/revoke/unauthorized/duplicate/invalid-ID)
Tue	Create Claim stepper (type select + details form)	/api/claims create endpoint, Claim schema wiring	Contract gas review, finalize on-chain vs off-chain field split with Dev B
Wed	Evidence upload UI (drag-drop, client validation)	Multer upload endpoint + evidenceValidator service (type/size/MIME/signature)	Web3Listener service skeleton (Ethers.js event subscription) in backend repo
Thu	Wire Create Claim → real /api/claims + evidence upload	hashService (SHA-256) + duplicate-hash check, wire into evidence pipeline	Connect Web3Listener to write verifications collection on ClaimVerified/ClaimRejected events
Fri	**Integration checkpoint**: full claim creation flow live (form → API → DB → hash	Same checkpoint from backend side; confirm claim lifecycle status transitions	Confirm event → DB sync working with a manual test transaction

	returned to UI)		

Week 2 Deliverable: Working profile and claim module — users can connect wallet, build a profile, create a claim, upload evidence, and see it hashed and stored.

Week 3 — Verification (the core workflow, heaviest week)

Day	Dev A	Dev B	Dev C
Mon	OTP modal UI (send/verify, countdown, resend cooldown)	`otpService` + `/api/auth/send-otp` + `/api/auth/verify-otp`, nodemailer setup	Verifier dashboard read APIs support: `/api/verifiers` list, sync registered verifiers on-chain ↔ off-chain
Tue	Verifier Dashboard UI (claim queue, expand/preview evidence)	`/api/verification/evidence-check` — runs evidenceValidator + hashService, updates status to EVIDENCE_CHECK/VERIFICATION_PENDING	Wire frontend Approve button to call contract `verifyClaim` via ethers (pair with Dev A)
Wed	Approve/Reject buttons → trigger MetaMask tx (pair with Dev C), tx pending/loading state	`trustScoreService`: point rules, caps, duplicate-verifier protection	Wire Reject button to `rejectClaim`; confirm revert cases surfaced cleanly in UI errors
Thu	Claim Details & Blockchain Proof page (self-claimed vs verified panels, tx hash link)	`/api/trust-score/:profileId` endpoint + `/api/verification/sync` to reconcile chain events into claim status	End-to-end test: create → OTP verify → evidence check → verifier approves in MetaMask → event fires → DB updates → UI reflects VERIFIED

Fri	**Integration checkpoint**: full workflow demo dry run (all 3 devs together)	Same	Same — fix any desync bugs between chain events and DB state

Week 3 Deliverable: End-to-end verification workflow — the entire "claim → evidence checks → verifier approval → blockchain proof" loop works live.

Week 4 — Testing + Demo Polish

Day	Dev A	Dev B	Dev C
Mon	Verification History timeline UI + Trust Dashboard (chart)	API tests for all endpoints (auth, profile, claims, verification) — Jest/Supertest	Finish Solidity edge-case tests: duplicate verification, invalid claim ID, unauthorized verifier revert cases
Tue	Admin/Verifier Management screen	Security pass: rate limiting on OTP/auth routes, input validation everywhere, file-upload hardening	Gas/security review of contract; confirm revocation preserves history (test with `queryFilter`)
Wed	UI polish pass: empty states, loading states, error toasts, mobile responsiveness	Integration test script: full flow scripted (wallet→profile→claim→OTP→evidence→verify)	Support Dev B on integration test; finalize deployment script for demo environment

Thu	Public profile share-link view (no-auth read-only) + accessibility/badge consistency check	Write README (API docs, setup steps, `.env.example`), architecture diagram doc	Write README (contract, testing, deployment steps), architecture diagram doc
Fri	**Final demo rehearsal** — run the exact Section 31 demo scenario end to end as a team	Same	Same

Week 4 Deliverable: Complete MVP + test suite + documentation + rehearsed live demo.

5. Testing Strategy Summary (owners)

Layer	**Owner**	**Covers**
Unit — OTP, hashing, claim validation, trust score, verification rules	Dev B	`services/*`
API — all listed endpoints (send-otp, verify-otp, claims CRUD, verification/request)	Dev B	Supertest/Jest against Express routes
Smart contract — create/verify/reject/revoke/unauthorized/duplicate/invalid-ID	Dev C	Hardhat + Mocha/Chai
Integration — full wallet→claim→evidence→verify→chain→profile flow	All 3 (Dev A drives UI clicks, Dev B/C verify backend/chain state)	Manual scripted run + optional Cypress/Playwright smoke test if time allows in Week 4

6. Demo

1. Connect MetaMask → 2. Create a "Python Developer" skill claim → 3. Upload evidence → 4. System validates file → 5. SHA-256 hash generated → 6. User verifies email via OTP → 7. Requests verification from a mentor → 8. Mentor opens verifier dashboard → 9. Reviews claim/evidence → 10. Clicks Verify → 11. MetaMask opens, mentor confirms → 12. TrustMesh.sol records verification → 13. Profile status → VERIFIED → 14. Recruiter opens public profile and sees verifier wallet, timestamp, evidence hash, and transaction hash.
